Scene Korean-Text Editing Based on STEFFAN

STEFFAN 기반의 장면 내 한국어 글자 편집

강민혜 [1], 김태호 [1], 이희재 [2], 고재필 [1]

금오공과대학교 컴퓨터 AI 융합공학과 [1], SNUAILAB[2]

gangminhye77@kumoh.ac.kr, taehokim@kumoh.ac.kr, huijae.lee@snuailab.ai, nonezero@kumoh.ac.kr,

요 약

이미지 내 글자를 수정하는 작업은 숙련된 편집 기술과 많은 시간이 요구된다. 최근 컴퓨터 비전의 발전으로 사람을 대신하여 원본 글자의 배경과 글꼴을 유지하면서 원하는 글씨로 변경하는 장면 내 글자 편집 기술이 연구되고 있다. 그러나 이는 대부분 영어를 기반으로 하여 표현 가짓수가 다양한 한국어에 바로 적용하는 것은 어렵다. 본 논문에서는 STEFANN 을 기반으로 장면 내 한국어 문자를 편집하는 방법론을 제안한다. 26 자의 영어 알파벳만을 변환하는 STEFANN 대비 완성형 한글을 포함한 2,412 자를 편집할 수 있으며, 전처리 개선 및 딥러닝 기반의 인페인팅의 적용으로 편집 결과물의 완성도를 향상시킨다. 실험 파트에서는 구축한 한국어 데이터셋으로 학습시킨 모델을 바탕을 실제 데이터에 적용한 예시를 보인다.

1. 서론

글자는 다양한 이미지 내에 존재한다. 다양한 이유로 이미지 내 글자의 편집이 필요하다. 그러나 복잡한 배경 속에서 글꼴, 색상, 크기를 유지하며 글자를 직접 편집하는 것은 숙련된 기술을 요구한다.

최근 심층 신경망의 발전과 함께 VAE(Variational Auto Encoder)[1], GAN(Generative Adversarial Nets))[2]과 같은 생성 모델이 등장했다. 이들은 사실적인 이미지 생성에 큰 도움을 주었다. 동시에 새로운 글꼴 합성에도 사용되었다. 관련 연구들은 두 개의 글자 이미지를 입력으로 받는다. 하나의 글자에서 글꼴을, 다른 글자에서 글씨를 갖고 새로운 글자를 합성한다. 그러나 이는 단조 색상의 배경에서만 수행된다. 또한 글자에 다양한 색상이 혼합되어 있거나, 카메라 위치에 따른 왜곡, 자연 조건에 따른 밝기, 대비, 그림자 등이 고려되지 않는 한계가 있다.

장면 내 글자 편집(Scene Text Editing) 기술은 원본 글자의 배경과 글꼴을 유지하면서 글씨를 원하는 글씨로 바꾸는 것을 목표로 한다. 이러한 목적을 달성하기 위해 심층 신경망 기반의 연구들이 진행되었다 [3, 4, 5]. 각 연구들은 모두 영어를 기반으로 진행되어 한국어에 바로 적용하기 어렵다.

본 논문에서는 STEFFAN(Scene Text Editor using Font Adaptive Neural Network)[3] 기반의 장면 내 한국어 글자 편집 방법론을 제안한다. 우리가 아는 한, 장면 내 글자 편집에 사용할 수 있는 공개 데이터 세트가 없다. 따라서 학습에 사용할 데이터 세트를 구축한다. 기존 방법론에서 사용한 방법은 영어에 최적화된 기법이다. 영어는 알파벳 26 개로 구성되어 있으며 대부분의 글자는 하나의 영역으로 이어져 있다. 그러나 현대 한국어 글자 수는 11,172 일뿐더러 하나의 글자에서 자음 자모가 떨어진 영역으로 나타나는 경우가 많다. 이러한 한글의 특성을 고려하여 적절하게 알고리즘을 개선한다.

2. 기존 장면 내 글자 편집 방법

본 장에서는 제안된 방법을 소개하기에 앞서, STEFFAN 에서 제안한 FANnet(Font Adaptive Neural Network), Colornet(Color Neural Network)과 각 모델의 입출력을 소개한다.

2.1 FANnet

FANnet 은 두 개의 입력을 받는다. 첫번째 입력은 64×64×1 크기의 이진 이미지 I_{ob}(original binary)이다. 다른 입력은 |V|크기의 원-핫 인코딩 벡터이다. |V|는 모델이 지원하는 수정가능한 글씨의 개수이다. 모델은 입력 이미지의 글꼴로 목표 글자 나타낸 64×64×1 크기의 이진 이미지 I_{eb}(edited binary)를 생성한다. 이는 I_{ob} 의 글꼴을 갖지만 V 가 가리키는 글씨를 나타낸다. 그림 1(a)는 FANnet 의 구조를 보이며, 두 입력 모두 512 크기의 벡터로 표현된 두 입력을 이어 붙인 것(concatenation)이 특징이다.

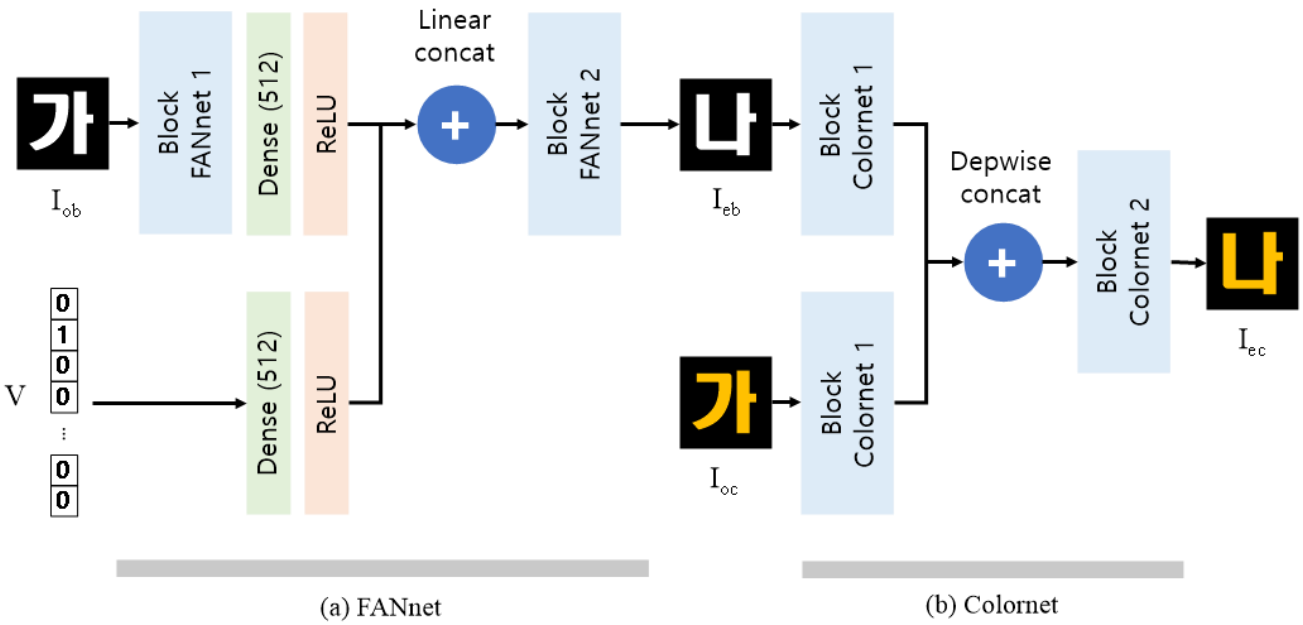


그림 1. (a) FANnet 의 구조 (b) Colornet 의 구조; 각 Block 은 콘볼루션, 완전 연결 레이어, 정규화, 활성화 함수 등으로 구성되어 있다. 자세한 구조와 인자는 STEFANN[3]을 참조

2.2 Colornet

Colornet 도 두 개의 입력을 받는다. 첫번째 입력은 FANnet 에서 생성된 I_{eb} 이다. 다른 입력은 I_{oc}(original color)로이미지 I 에 I_{ob} 를 마스킹하여 획득한 64×64×3 크기의 이미지이다. 모델은 64×64×3 크기의 이미지 I_{ec}(edited color)를 생성한다. 이는 I_{eb} 와 동일한 글자를 생성하며 이는 I_{oc} 의 색상으로 표현된다. 그림 1(b)는 Colornet 의 구조를 보이며, 두 입력은 각각의 콘볼루션 레이어를 지난 후 채널별로 연결된다. 그림 2 는 원본 이미지와 입출력 이미지 I_{ob}, I_{oc}, I_{eb}, I_{ec} 의 예시를 보인다.

그림 2. 네트워크의 입출력 이미지 예시; (a)는 원본 영상, (b)는 원본의 이진 영상, (c)는 I 와 I_{ob} 의 논리곱을 통해 생성된 영상이다. (d)와 (e)는 각각 FANnet 과 Colornet 의 출력 결과이다.

3. 제안된 방법론

3.1 데이터세트 구축

FANnet 을 학습시키기 위한 데이터 세트는 검은색 배경에 다양한 글꼴을 갖는 이진 글자 이미지로 구성된다. 하나의 글꼴에서 선택된 두 글씨와 그 중 한 글씨의 원-핫 인코딩 벡터를 생성하여 입력쌍으로 사용한다. 글꼴은 한국저작권위원회가 운영하는 공유 마당의 안심 글꼴파일 서비스[6]를 통해 총 14 개의 폰트를 수집하였다. 이진 이미지는 검은 이미지 배경의 원하는 폰트로 생성하고자 하는 문자를 그려 생성한다. 이 때, 백지 크기와 글자 크기는 400, 128 을 적용하여 충분히 큰 글자 이미지를 생성한다. 글꼴마다 실제 화면 그려지는 크기에 차이가 있기 때문에 글자를 포함하는 가장 작은 바운드 박스를 찾고 일정한 크기 및 위치를 조정한다. 그림 3(a) 구축된 FANnet 데이터세트를 보인다.

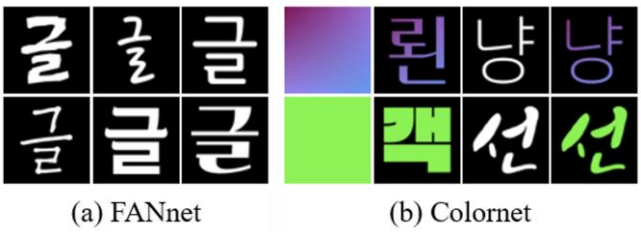


그림 3. 데이터 세트 예시

Colornet 을 학습시키기 위해서는 입력으로 사용되는 이진 글자 이미지와 색상 글자 이미지, 목표 이미지가 필요하다. 이진 글자 이미지는 FANnet 을

위해 생성한 데이터를 사용한다. 색상 글자 이미지를 생성하기 위해 64×64 크기의 단조 색상과 그라데이션으로 구성된 배경색 이미지를 준비한다. 색상 글자 이미지는 배경색 이미지와 이진 글자 이미지의 마스킹을 통해 생성할 수 있다. 그림 3(b)는 Colornet 데이터세트를 보인다.

3.2 파이프라인

제안하는 방법론의 파이프라인은 글자 합성, 색상 전이, 글자 배치 3 단계로 구성된다. 그림 4 는 전체 파이프라인을 보인다.

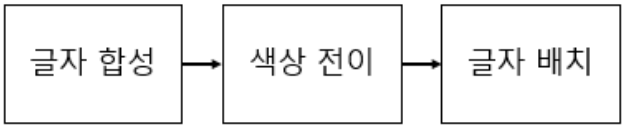


그림 4. 전체 파이프라인

글자 합성 단계는 FANnet 을 통해 새로운 글자를 생성하는 단계이다. FANnet 의 입력으로는 이진 글자 이미지와 원-핫 인코딩 된 글씨 벡터가 입력으로 사용된다. 그러나 자연에서 촬영된 영상은 이진 영상이 아니다. STEFANN 에서는 MSER 알고리즘을 기반하여 이진화를 수행하였으나 한글 글자의 자음 모음을 각각 분리하여 글자 영역을 찾아내기에 한글에 적용하기에는 어렵다. 따라서 적응형 이진화(Adaptive Threshold) 알고리즘을 통해 이진화를 수행한다. FANnet 에서 사용하는 이진 이미지는 배경이 검은색이고 글자가 흰색이다. 적응형 이진화의 결과물은 배경과 전경색에 따라서 그 결과가 반대일 수 있다. 이를 보정하기 위해서 이진화 영상 테두리의 색 개수를 확인한다. 테두리는 배경이라는 가정하에 흰색의 개수가 많다면 색반전을 수행한다. 그림 5 는 배경과 전경의 픽셀 개수의 차이에 따라 이진화 결과가 반전되는 예시를 보인다. 이 후 FANnet 의 입력 크기인 64×64 로 크기 변환을 수행한다. 학습 데이터 세트와 유사한 조건을 갖추기 위해 글자를 포함하는 바운딩 박스를 찾고, 50×50 으로 크기 변환 후 7 만큼의 패딩을 추가한다. 이후 FANnet 에 글씨 벡터와 함께 입력으로 사용하여 이진 글자 이미지를 생성한다. 생성된 이진 글자 이미지는 완전한 이진 이미지가 아니므로 별도의 이진화를 수행한다.

(a) 원본 이미지 (b) 이진화 이미지

그림 5. 이미지 이진화 시 반전 예시

색상 전이 단계에서는 Colornet 을 통해, 이전 단계에서 생성된 이미지에 원본 글자의 색상을 입힌다. 생성된 이미지의 배경색은 완전한 검은색이 아니다. 따라서 이전 단계에서 생성된 영상과 논리곱 연산을 수행한다.

글자 배치 단계는 인페인팅 기술을 통해 기존 글자를 지운 후, 이전 단계에서 생성된 글자를 합성한다.

4. 실험 결과 및 분석

실험에서는 우리가 훈련한 모델의 중간 결과물인 글꼴 변환 영상과 글자 색상 변환 영상, 그리고 실사 데이터 적용한 예시 영상을 보여준다. 데이터세트는 학습용 글꼴 11 개와 검증용 글꼴 3 개를 사용하였으며 이를 2412 개의 글씨(완성형 한국어 글씨 2350 개, 알파벳 52 개, 아라비아 숫자 10 개)와 조합하였다. FANnet 에서 생성될 수 있는 총 입력 쌍의 개수는 약 1 억개이다. 모든 데이터 쌍을 학습에 사용할 경우 학습속도가 저하되어 매 에폭마다 100 만 개의 쌍을 선택하여 학습하였다. Colornet 을 학습하기 위해서 별도의 배경색 이미지를 학습용 800 개, 검증용 200 개를 사용하였다. 위와 유사하게, 생성될 수 있는 총 입력 쌍의 개수가 방대하여 매 2336 개의 쌍을 선택하여 학습하였다.

FANnet 은 최적화 알고리즘은 Adam, 학습률 5e-4, L1 로스함수를 사용하였으며 100 에폭 학습하였다. Colornet 은 FANnet 과 동일한 파라미터에서 학습하였다. 그림 6 은 학습 및 검증 로스 그래프이다.

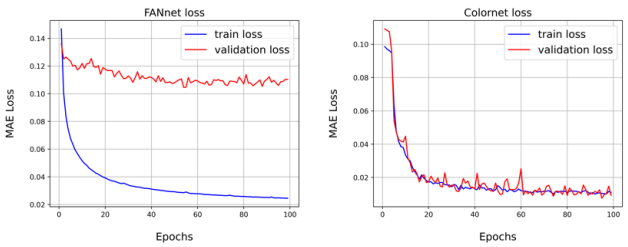


그림 6. 학습 및 검증 로스 그래프

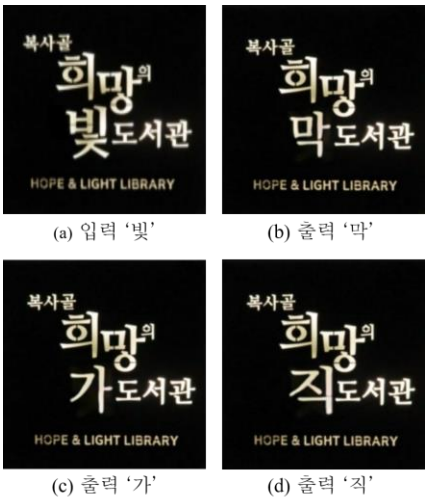
FANnet 은 학습 초반에는 얇은 폰트의 생성이 잘 되지 않았으나 점차 자연스럽게 생성되었다. 굵은 폰트의 경우 일부 폰트 내에 강한 개성을 가진 폰트로만 출력되는 경향이 보였다. Colornet 은 초반에는 아무것도 출력하지 못하다가 점차 단색 및 그라데이션의 색상이 출력되었고 후반에는 색상까지 비슷하게 출력되는 것을 볼 수 있다.

그림 8 과 9 는 본 논문에서 제안된 방법으로 편집한 글자 이미지 예시이다. 그림 8(a)는 입력 이미지이며 그림 8(b)는 입력의 글자 '정'을 '통'으로 수정한 이미지이다. 그림 9(a)는 입력 이미지이며 9(b)는 '빛'을 '막'으로, 9(c)는 '가'로, 9(d)는 '직'으로 수정하였다. 생성된 이미지가 글자의 기울임을 반영하지 못하고 복잡한 배경이 조금 부자연스럽지만, 원본 글자의 글씨와 색상을 자연스럽게 반영하여 생성된 것을 확인할 수 있다.

그림 7. 에폭 별 생성한 결과물

(a) 입력 '정' (b) 출력 '통'

그림 8. 실제 이미지 글자 편집 예시 1



(a) 입력 '빛' (b) 출력 '막'

(c) 출력 '가' (d) 출력 '직'

그림 9. 실제 이미지 글자 편집 예시 2

5. 결론

본 논문에서는 STEFANN 을 기반으로 한 장면 내 한국어 글자 편집 시스템을 구현하였다. 영어 기반의 기존 연구를 개선하여 한국어에도 글꼴과 글자의 색을 모두 적절하게 반영하여 편집된 이미지를 생성할 수 있음을 보였다.

그러나 이러한 방법론은 입력으로 학습하지 않은 완성형 한글 외의 글자를 생성할 수 없으며 글자의 기하학적 구조를 반영하지 못하고 여전히 조금 부자연스러운 부분이 존재한다. 추후 연구에서는 이러한 요구사항들을 해결할 수 있는 SwapText[4]를 개선하여 진행할 예정이다.

Acknowledgement

본 논문은 2022 년 중소벤처기업부의 기술개발사업지원(G26S310751502)에 의해 수행되었습니다.

참고문헌

[1] Kingma, D. P., & Welling, M., "Auto-encoding variational bayes," ICLR., 2014

[2] Goodfellow, Lan J., et al., "Generative Adversarial Nets," NIPS, 2014

[3] Roy, Parsun, et al., "STEFANN: Scene text editor using font adaptive neural network," Proceddings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020

[4] Yang, Q., Huang, J., & Lin, W., "Swaptext: Image based texts transfer in scenes," Proceddings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition, 2020

[5] Qu, Yadong, et al., "Exploring Stroke-Level Modification for Scene Text Editing," arXiv preprint, 2022

[6] 한국저작권위원회 공유마당 민간 안심글꼴, "https://gongu.copyright.or.kr/gongu/singl/wrtContent/freeFont.do?menuNo=200323"